

Informe Técnico del Sistema Backend "Biblioteca API"

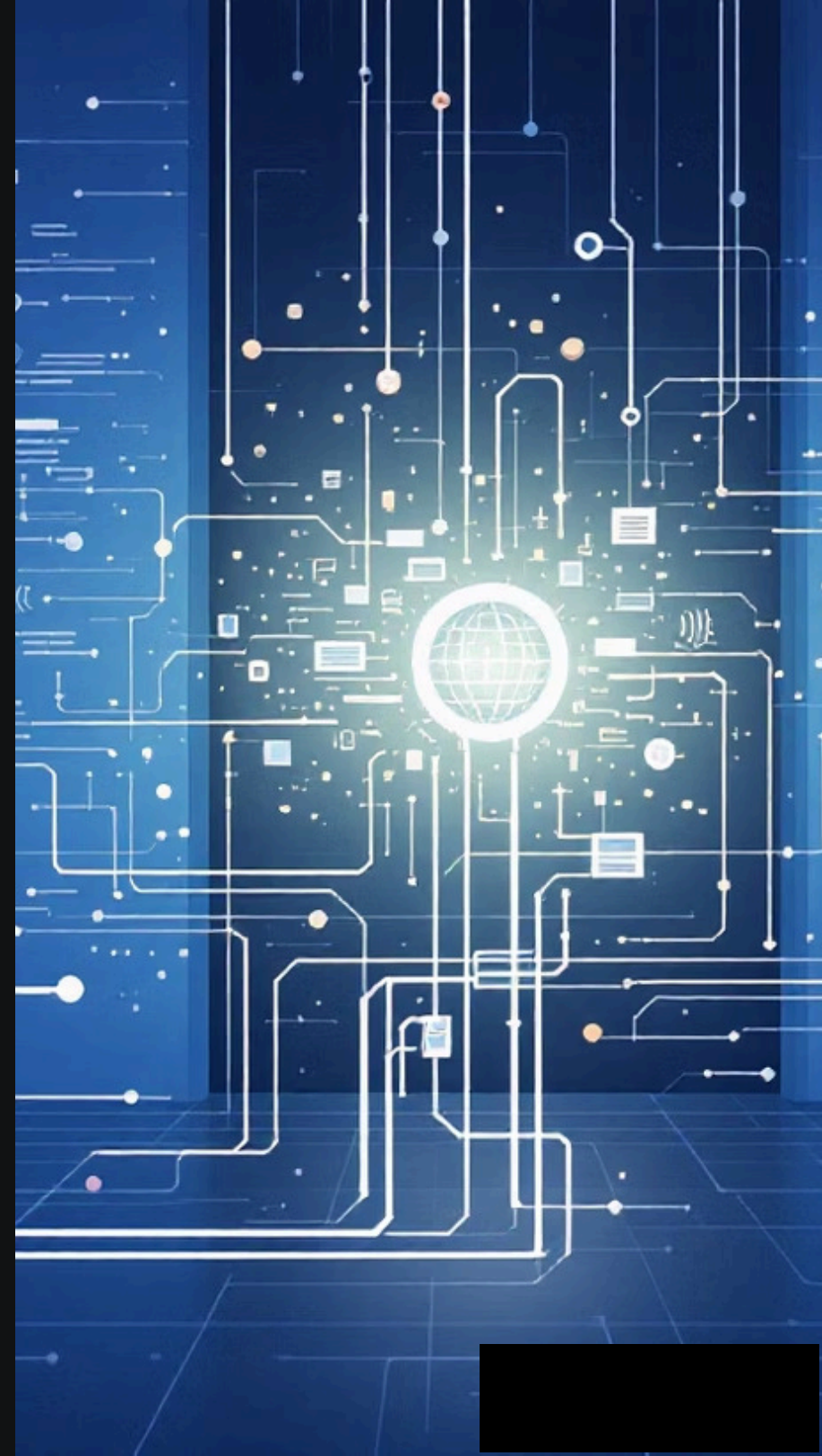
Arquitectura, Seguridad y Persistencia

Proyecto desarrollado con Java 21, Spring Boot, Spring Security, JWT, JPA/Hibernate y MySQL

Un sistema robusto de gestión bibliotecaria con arquitectura por capas, autenticación segura y control de acceso basado en roles

Integrantes:

1. Velarde Robles Francisco Xavier Leon
2. Roman Huaman Josled Luis Antonio
3. Peña Chavez Gissel Melani
4. Osorio Guzman Jose Luis
5. Colina Martin Jesus Gabriel





Ejecución del Proyecto – Biblioteca API (Backend + Frontend) Backend – Spring Boot (Java 21)

1- Requisitos  Java 17+ |  Maven 3+ |  MySQL 8+ |  IDE (IntelliJ / VSCode)

2- Base de Datos o se crea automaticamente

CREATE DATABASE biblioteca;

3- Configuración (application.properties)

URL BD, usuario, contraseña

spring.jpa.hibernate.ddl-auto=update

4- Ejecutar Backend

IDE → Run 

Terminal → mvn spring-boot:run

5- Swagger (API Docs)  <http://localhost:8080/swagger-ui/index.html>

6- Autenticación

- Registrar usuario
- Login → obtener JWT
- Swagger → Authorize → Bearer {token}



Frontend – (React / Angular / Vue)

1- Requisitos  Node.js + npm

2- Instalar dependencias

npm install

Ejecutar Frontend

-npm run dev

 Abrir en navegador: <http://localhost:5173>

(o el puerto indicado)

3- Consumo de la API

Llamadas a endpoints protegidos con Bearer Token

Manejo de roles en interfaz (ADMIN / USER)



Funciones Principales (Back + Front)

-  Gestión de libros (CRUD + soft delete)
-  Préstamos y devoluciones
-  Usuarios, roles y autenticación JWT
-  Interfaz gráfica para consumir API
-  Validaciones, mensajes de error y control de acceso

Resumen Ejecutivo

El sistema **Biblioteca API** es una solución backend completa desarrollada para gestionar eficientemente los recursos y operaciones de una biblioteca moderna. Implementado con tecnologías empresariales de última generación, el sistema proporciona funcionalidades robustas de gestión de libros, usuarios y préstamos.

Objetivo del Sistema

Proporcionar una plataforma segura y escalable para la gestión integral de bibliotecas, incluyendo control de inventario, administración de usuarios y seguimiento de préstamos

Funcionalidades Principales

- Gestión completa de libros con soft delete
- Sistema de préstamos y devoluciones
- Autenticación JWT con roles
- Control de acceso granular

Tecnologías Utilizadas

- Java 21 y Spring Boot
- Spring Security con JWT
- JPA/Hibernate y MySQL
- Swagger/OpenAPI

Introducción

Contexto del Problema

Las bibliotecas modernas enfrentan desafíos significativos en la gestión manual de sus recursos. La necesidad de un sistema de gestión de biblioteca surge de la complejidad inherente al control de inventarios extensos, el seguimiento de préstamos, la administración de usuarios y la generación de reportes precisos.

Necesidad de un Sistema de Gestión de Biblioteca

Un sistema automatizado permite:

- Reducir errores humanos en el registro
- Agilizar procesos de préstamo y devolución
- Mantener trazabilidad completa de operaciones
- Generar estadísticas en tiempo real
- Mejorar la experiencia del usuario
- Optimizar recursos administrativos

Objetivos del Proyecto

Objetivo General:

Desarrollar un sistema backend robusto y seguro para la gestión integral de bibliotecas utilizando arquitectura por capas y mejores prácticas de desarrollo.

Objetivos Específicos:

1. Implementar arquitectura MVC escalable
2. Garantizar seguridad mediante JWT
3. Diseñar modelo de datos normalizado
4. Proporcionar API REST documentada

Arquitectura General del Sistema

Arquitectura por Capas

El sistema implementa una **arquitectura por capas** siguiendo el patrón MVC (Model-View-Controller), garantizando separación de responsabilidades, mantenibilidad y escalabilidad.

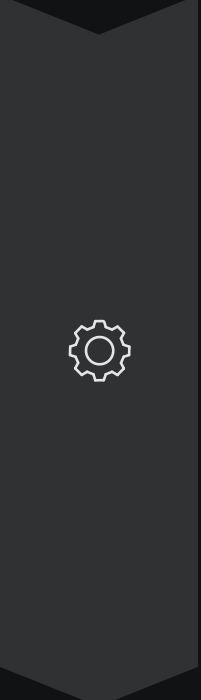


1. Capa de Presentación (Controller)

Responsable de:

- Exponer **endpoints REST**
- Recibir y validar peticiones (@Valid)
- Convertir DTOs de entrada en objetos de dominio
- Retornar respuestas JSON estructuradas
- Delegar operaciones en la capa de servicios

Nunca contiene lógica de negocio.

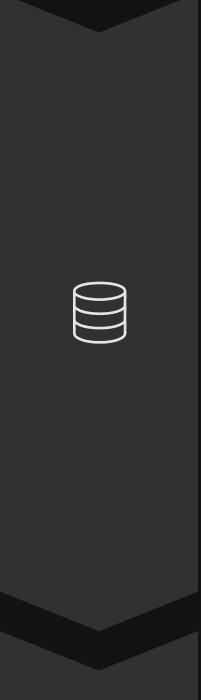


2. Capa de Negocio (Service)

Responsable de:

- Contener la **lógica de negocio real**
- Aplicar reglas del dominio
- Orquestar operaciones entre repositorios
- Validar estados del sistema (ej.: disponibilidad de libros)
- Manejar transacciones (@Transactional)

Es la capa más crítica del sistema.



3. Capa de Persistencia (Repository)

Responsabilidades:

- Abstraer acceso a la base de datos
- Proveer operaciones CRUD mediante **Spring Data JPA**
- Ejecutar consultas personalizadas derivadas del nombre
- Gestionar relaciones entre entidades

Existen repositorios para Usuario, Libro, Rol y Prestamo.

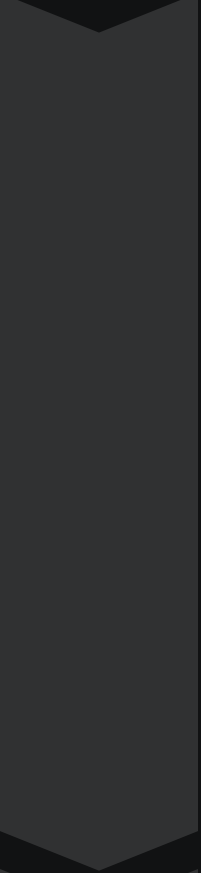


4. Capa de Modelo de Datos (Entity)

Incluye:

- Entidades JPA
- Relaciones complejas (OneToMany, ManyToMany, ManyToOne)
- Mapea tablas y columnas en MySQL
- Validaciones estructurales (@NotNull, @Size, etc.)

Representa la base del dominio.



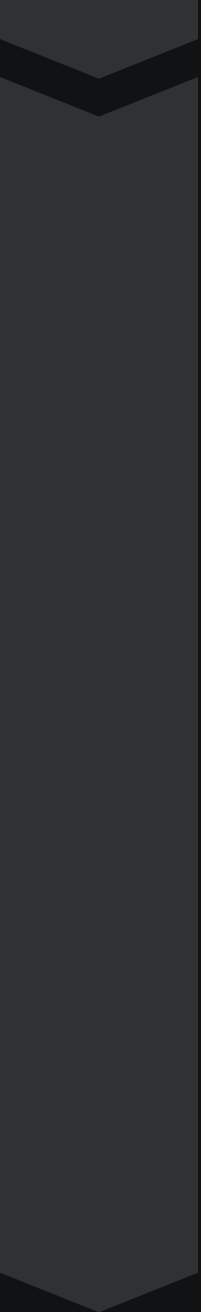
5. Capa de Seguridad (Security)

Es una **capa transversal** que protege el sistema completo.

Componentes principales:

- JwtAuthenticationFilter** → intercepta y valida tokens
- SecurityConfig** → define reglas de acceso por rol
- JwtService** → genera y valida tokens
- UserDetailsService** → carga usuarios desde BD
- AuthenticationEntryPoint** y **AccessDeniedHandler** → manejan 401 y 403

Provee autenticación **stateless** mediante tokens JWT firmados.



6. Capa de Transferencia de Datos (DTOs + Mappers)

Esta capa facilita la comunicación entre cliente y servidor.

Incluye:

DTOs (Data Transfer Objects)

- Evitan exponer entidades directamente
- Validan datos de entrada
- Estandarizan respuestas

Mappers (MapStruct)

- Transforman Entity ↔ DTO automáticamente
- Mejoran mantenibilidad
- Reducen código repetitivo

Esta capa está presente en todos los flujos del sistema.

Flujo de Petición Cliente → Servidor → BD → Respuesta

Cada petición sigue un flujo estructurado:

- Cliente** envía petición HTTP al endpoint REST
- JwtAuthenticationFilter** intercepta y valida el token JWT
- SecurityConfig** verifica permisos del rol para autorizar el acceso al endpoint.
- Controller** recibe la petición y valida los datos usando DTOs con @Valid.
- Mapper** convierte el **DTO** recibido en una entidad del dominio.
- Service** aplica la lógica de negocio y verifica reglas del sistema.
- Service** coordina operaciones con los Repositories de JPA.
- Repository** ejecuta consultas CRUD traducidas por Hibernate.
- Hibernate** genera el SQL correspondiente y lo envía a MySQL.
- MySQL** procesa la consulta y retorna los datos solicitados.
- Mapper** convierte la Entidad en un DTO de respuesta.
- Controller** envía la respuesta JSON al cliente, y GlobalExceptionHandler formatea errores si ocurren.



Capa de Controladores: Arquitectura REST

La capa de controladores constituye la interfaz principal entre cliente y servidor, exponiendo endpoints HTTP y delegando operaciones a la capa de servicios. Implementa un diseño RESTful con separación clara de responsabilidades.



AuthController

Gestiona autenticación y registro de usuarios mediante JWT con validación de credenciales y tokens de 24 horas



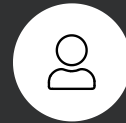
LibroController

Controla operaciones CRUD completas sobre libros con soft delete y control de disponibilidad



PrestamoController

Administra préstamos y devoluciones con validación de disponibilidad y estados transaccionales



UsuarioController

Permite consulta y administración de usuarios con control de acceso basado en roles



AuthController: Seguridad y Autenticación

Endpoints de Autenticación

POST /auth/register

Registra nuevos usuarios en el sistema con validaciones exhaustivas sobre email único y contraseña segura. Delega el proceso a `AuthService.register()` que cifra credenciales con `BCrypt`.

POST /auth/login

Autentica usuarios mediante credenciales y genera tokens JWT válidos por 24 horas. Implementa `AuthService.login()` con validación contra base de datos y generación de claims personalizados.

Responsabilidades Clave

- Validación de DTOs con anotaciones `@Valid` para registro y login
- Generación de tokens JWT con algoritmo HS256 y secret key configurable
- Cifrado de contraseñas mediante `BCryptPasswordEncoder`
- Prevención de exposición de información sensible en respuestas
- Manejo de excepciones de credenciales inválidas

LibroController: Gestión Integral de Recursos

Controla el ciclo de vida completo de los recursos bibliográficos con validaciones robustas, control de permisos diferenciado por rol y operaciones de soft delete para mantener integridad referencial.

01

Consulta de Catálogo

GET /libros lista todos los libros activos. GET /libros/{id} obtiene detalles específicos con validación de existencia.

02

Creación (Solo ADMIN)

POST /libros valida ISBN único, cantidad mínima de ejemplares y campos obligatorios mediante @NotBlank y @NotNull.

03

Actualización

PUT /libros/{id} permite modificar información bibliográfica manteniendo validaciones de integridad y unicidad de ISBN.

04

Eliminación Lógica (Solo ADMIN)

DELETE /libros/{id} ejecuta soft delete cambiando estado a inactivo. PUT /libros/{id}/activar permite reactivación.



Seguridad por Rol: Los usuarios con rol USER tienen acceso exclusivamente de lectura al catálogo, mientras que ADMIN posee privilegios completos para todas las operaciones CRUD y gestión de estados.

PrestamoController: Gestión de Transacciones

Operaciones Principales

1

Crear Préstamo

POST /prestamos/admin verifica disponibilidad de ejemplares y crea registro transaccional con estado ACTIVO, decrementando contador de libros disponibles.

2

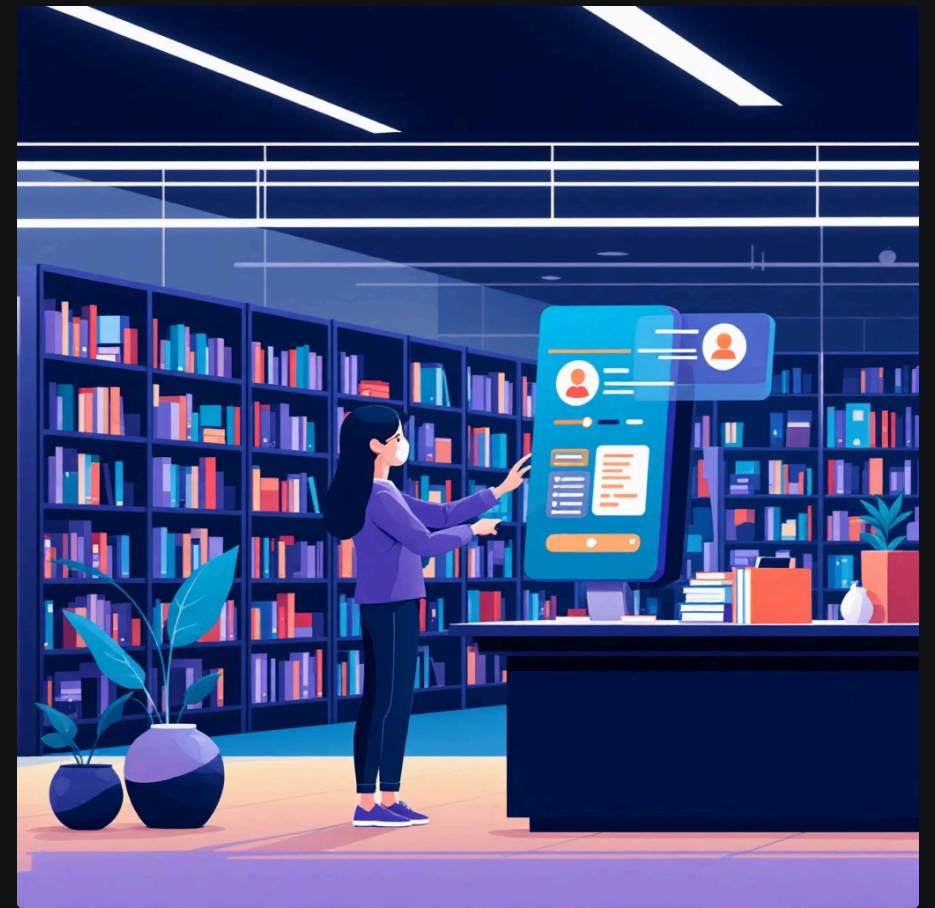
Registrar Devolución

PUT /prestamos/{id}/devolver actualiza estado a DEVUELTO, registra fecha de devolución e incrementa ejemplares disponibles del libro.

3

Consultas Filtradas

GET /prestamos/mios muestra historial del usuario autenticado. GET /prestamos/admin permite auditoría completa.



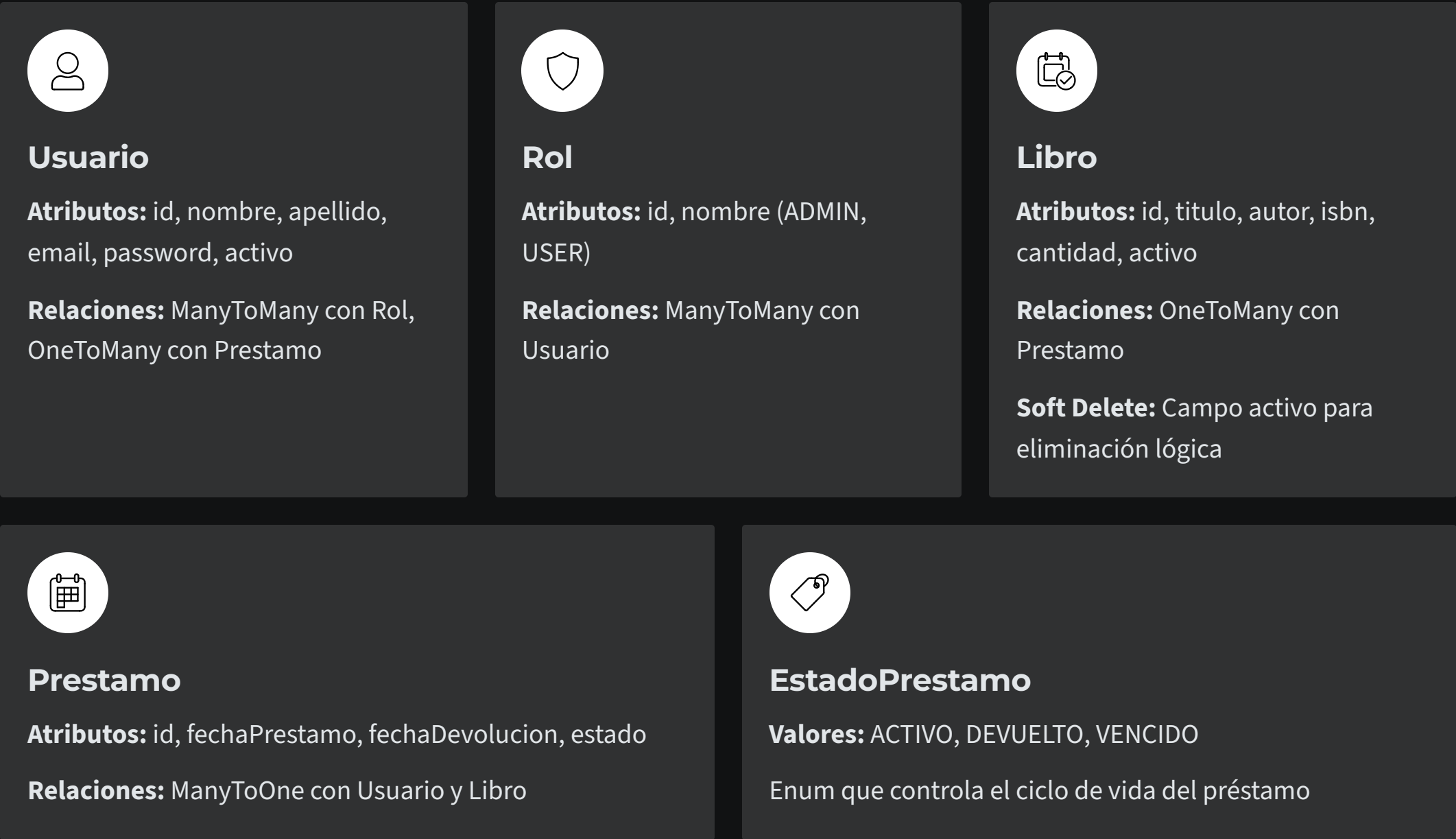
Reglas de Negocio

- Validación de disponibilidad antes de crear préstamo
- Actualización atómica de contadores de ejemplares
- Transición de estados: ACTIVO → DEVUELTO
- Control de propiedad: usuarios solo pueden ver sus préstamos
- Registro de timestamps para auditoría completa

Diseño e Implementación del Modelo de Datos

Diagrama Entidad-Relación

El modelo de datos está diseñado siguiendo principios de normalización y representa las entidades principales del dominio bibliotecario.



Uso de Anotaciones JPA

Anotaciones de Entidad:

- @Entity - Define clase como entidad JPA
- @Table - Especifica nombre de tabla
- @Id - Marca clave primaria
- @GeneratedValue - Estrategia de generación de ID
- @Column - Configura columnas específicas

Anotaciones de Relación:

- @ManyToMany - Relación muchos a muchos (Usuario-Rol)
- @OneToMany - Relación uno a muchos (Libro-Prestamo)
- @ManyToOne - Relación muchos a uno (Prestamo-Usuario)
- @JoinTable - Configura tabla intermedia
- @JoinColumn - Define columna de unión

Validaciones con Bean Validation

El sistema implementa validaciones robustas utilizando anotaciones estándar:

- @NotBlank - Valida que campos de texto no estén vacíos (nombre, apellido, titulo)
- @Email - Verifica formato válido de correo electrónico
- @Min - Establece valor mínimo para campos numéricos (cantidad de libros)
- @NotNull - Garantiza que campos obligatorios no sean nulos
- @Size - Controla longitud de cadenas (password mínimo 6 caracteres)

Normalización del Modelo

El modelo cumple con la **Tercera Forma Normal (3FN)**, eliminando redundancias y garantizando integridad referencial mediante claves foráneas y restricciones de base de datos.

Persistencia con JPA / Hibernate

Uso de Spring Data JPA

Spring Data JPA simplifica drásticamente el acceso a datos mediante la abstracción de repositorios, eliminando código boilerplate y proporcionando consultas derivadas automáticamente.

Repositorios y Consultas Personalizadas Cada entidad cuenta con su repositorio que extiende <code>JpaRepository</code>: <code>UsuarioRepository.findByEmail(String email)</code> - Búsqueda por email único <code>LibroRepository.findByActivoTrue()</code> - Obtiene solo libros activos <code>PrestamoRepository.findByUsuarioAndEstado()</code> - Filtra préstamos por usuario y estado <code>RolRepository.findByNombre(String nombre)</code> - Búsqueda de roles por nombre Estas consultas se generan automáticamente por Spring Data JPA interpretando el nombre del método.	Transaccionalidad con @Transactional La anotación <code>@Transactional</code> garantiza atomicidad en operaciones complejas: - Todas las operaciones en métodos de servicio son transaccionales - Si ocurre una excepción, se realiza rollback automático - Garantiza consistencia en operaciones que afectan múltiples tablas - Ejemplo: Al crear un préstamo, se actualiza la cantidad de libros disponibles en una sola transacción	Manejo de Datos en Cascada Las operaciones en cascada se configuran estratégicamente: <code>CascadeType.PERSIST</code> - Al guardar un usuario, se persisten sus roles <code>CascadeType.MERGE</code> - Actualización en cascada de entidades relacionadas <code>orphanRemoval = true</code> - Elimina préstamos huérfanos automáticamente - Evita <code>CascadeType.REMOVE</code> en relaciones críticas para prevenir eliminaciones accidentales
--	---	---

Optimización del Acceso a Datos

- Lazy Loading:** Carga diferida de colecciones para mejorar rendimiento
- Fetch Joins:** Consultas optimizadas para evitar N+1 queries
- Índices:** Campos como email e isbn tienen índices únicos
- Paginación:** Uso de `Pageable` para grandes conjuntos de datos
- Caché de segundo nivel:** Configuración de Hibernate para entidades frecuentes

Justificación del Uso de Hibernate

- Hibernate** fue seleccionado como proveedor JPA por:
- Madurez y estabilidad comprobada en producción
 - Excelente documentación y comunidad activa
 - Optimizaciones automáticas de consultas SQL
 - Soporte completo de características JPA 2.2
 - Herramientas de debugging y logging avanzadas
 - Compatibilidad con múltiples bases de datos

Seguridad y Autenticación

Spring Security + JWT

El sistema implementa un **robusto mecanismo de seguridad** basado en JSON Web Tokens (JWT) y Spring Security, garantizando autenticación stateless y control de acceso granular basado en roles.

Arquitectura de Seguridad

La arquitectura de seguridad se compone de múltiples capas que trabajan en conjunto:

01

Autenticación Inicial

El usuario envía credenciales (email/password) al endpoint `/auth/login`

02

Validación de Credenciales

AuthService verifica credenciales contra la base de datos usando BCrypt para comparar passwords hasheados

03

Generación de Token JWT

JwtService genera un token firmado con algoritmo HS256 incluyendo claims: email, roles, fecha de expiración

04

Retorno del Token

El token se retorna al cliente quien debe incluirlo en el header Authorization de peticiones subsecuentes

05

Validación en Cada Petición

JwtAuthenticationFilter intercepta peticiones, extrae y valida el token, establece contexto de seguridad

06

Control de Acceso

Spring Security verifica que el usuario autenticado tenga los roles necesarios para acceder al recurso solicitado

Manejo Global de Excepciones

El sistema implementa un mecanismo centralizado mediante `@RestControllerAdvice` que intercepta todas las excepciones, estandariza respuestas de error y facilita debugging sin exponer detalles internos de implementación.



MethodArgumentNotValidException

Captura errores de validación en DTOs retornando mensajes descriptivos por campo con código 400 BAD_REQUEST.



EntityNotFoundException

Maneja recursos no encontrados retornando 404 NOT_FOUND con mensaje contextual sobre la entidad solicitada.



BadCredentialsException

Intercepta fallos de autenticación retornando 401 UNAUTHORIZED sin revelar si el usuario existe o la contraseña es incorrecta.



AccessDeniedException

Gestiona intentos de acceso sin permisos adecuados retornando 403 FORBIDDEN con indicación de rol requerido.



ExpiredJwtException

Detecta tokens JWT expirados retornando 401 con instrucción de renovar autenticación para continuar operaciones.



IllegalStateException

Captura violaciones de reglas de negocio como préstamo sin disponibilidad, retornando 400 con mensaje específico del conflicto.

Componentes de Seguridad - Análisis Detallado



JwtService

Responsabilidades principales:

- `generateToken(UserDetails)` - Genera JWT con claims personalizados incluyendo email y roles
- `extractUsername(String token)` - Extrae el email del usuario del token
- `validateToken(String token, UserDetails)` - Verifica firma, expiración y validez del token
- `extractClaim(String token, Function resolver)` - Extrae claims específicos del payload

Utiliza una clave secreta configurada en `application.properties` y establece tiempo de expiración de 24 horas.



JwtAuthenticationFilter

Filtro personalizado que intercepta todas las peticiones HTTP:

- Extiende `OncePerRequestFilter` para garantizar ejecución única por petición
- Extrae token del header `Authorization: Bearer {token}`
- Valida formato y firma del token usando `JwtService`
- Carga detalles del usuario mediante `CustomUserDetailsService`
- Establece `SecurityContextHolder` con autenticación válida
- Permite continuar la cadena de filtros si la validación es exitosa



CustomUserDetailsService

Implementa `UserDetailsService` de Spring Security:

- Método `loadUserByUsername(String email)` busca usuario en BD
- Convierte entidad Usuario a objeto `UserDetails` de Spring Security
- Mapea roles de la entidad a `GrantedAuthority`
- Lanza `UsernameNotFoundException` si el usuario no existe
- Verifica que el usuario esté activo antes de retornar detalles



CustomAccessDeniedHandler

Maneja errores de acceso denegado (403 Forbidden):

- Se activa cuando un usuario autenticado intenta acceder a recurso sin permisos
- Retorna respuesta JSON estructurada con código 403
- Incluye mensaje descriptivo: "No tienes permisos para acceder a este recurso"
- Registra intentos de acceso no autorizado en logs para auditoría



CustomAuthenticationEntryPoint

Maneja errores de autenticación (401 Unauthorized):

- Se activa cuando se intenta acceder a recurso protegido sin token válido
- Retorna respuesta JSON con código 401
- Mensaje: "Debes autenticarte para acceder a este recurso"
- Previene exposición de información sensible en mensajes de error

Control de Acceso Según Rol

El sistema implementa dos roles principales con permisos diferenciados:

Rol ADMIN

- Acceso completo a gestión de usuarios
- Crear, editar y eliminar libros
- Activar/desactivar libros (soft delete)
- Ver todos los préstamos del sistema
- Gestionar roles de usuarios
- Acceso a endpoints de administración
- Devolver libros prestados

Rol USER

- Ver catálogo de libros activos
- Ver historial personal de préstamos
- Acceso limitado a endpoints públicos y propios

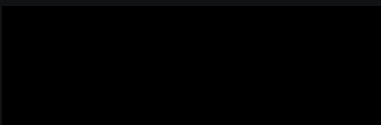
Protección Granular de Endpoints

La configuración de seguridad utiliza `SecurityFilterChain` para definir reglas precisas:

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
    .requestMatchers(HttpMethod.POST, "/api/libros/**").hasRole("ADMIN")
    .requestMatchers(HttpMethod.DELETE, "/api/libros/**").hasRole("ADMIN")
    .requestMatchers(HttpMethod.GET, "/api/usuarios/**").hasRole("ADMIN")
    .anyRequest().authenticated()
);
```

Por Qué el Sistema es Seguro

- Autenticación Stateless:** No se mantiene sesión en servidor, escalabilidad horizontal
- Tokens Firmados:** JWT firmado con HS256 previene manipulación
- Passwords Hasheados:** BCrypt con salt automático, nunca se almacenan en texto plano
- Expiración de Tokens:** Tokens tienen vida útil limitada (24h)
- Validación en Cada Petición:** Verificación continua de autenticidad
- Principio de Menor Privilegio:** Usuarios solo acceden a recursos necesarios
- Manejo Seguro de Errores:** No se expone información sensible en mensajes
- CORS Configurado:** Control de orígenes permitidos



Control de Acceso por Roles y Documentación Swagger

Matriz de Permisos por Endpoint

Endpoint	Método	Descripción	Rol Requerido	Códigos
/auth/register	POST	Registro de nuevos usuarios	Público	201, 400
/auth/login	POST	Autenticación y generación de JWT	Público	200, 401
/libros	GET	Listar todos los libros activos	USER, ADMIN	200
/libros/{id}	GET	Obtener detalles de un libro	USER, ADMIN	200, 404
/libros	POST	Crear nuevo libro	ADMIN	201, 400
/libros/{id}	PUT	Actualizar información de libro	ADMIN	200, 404
/libros/{id}/activar	PUT	Activar libro (revertir soft delete)	ADMIN	200, 404
/libros/{id}	DELETE	Soft delete de libro	ADMIN	204, 404
/prestamos/admin	POST	Crear nuevo préstamo	ADMIN	201, 400
/prestamos/{id}/devolver	PUT	Registrar devolución de libro	ADMIN	200, 404
/prestamos/mis-prestamos	GET	Consulta préstamos del usuario autenticado	USER/ADMIN	200
/prestamos/admin	GET	Listar todos los préstamos	ADMIN	200
/usuarios	GET	Listar todos los usuarios	ADMIN	200, 404
/usuarios/{id}	GET	Obtener detalles de usuario	ADMIN	200, 404
/usuarios/email/{email}	GET	Busca usuario por dirección de correo electrónico	ADMIN	200, 404

Documentación con Swagger

Qué es OpenAPI

OpenAPI (anteriormente Swagger) es una especificación estándar para describir APIs REST de forma legible tanto para humanos como para máquinas. El sistema utiliza **SpringDoc OpenAPI 3** para generar documentación interactiva automáticamente.

Rutas Configuradas

- /swagger-ui/index.html - Interfaz gráfica interactiva
- /v3/api-docs - Especificación OpenAPI en JSON
- /v3/api-docs.yaml - Especificación en formato YAML

Cómo se Agrega el Token en Swagger

La configuración incluye un esquema de seguridad Bearer JWT:

- Hacer clic en el botón **"Authorize"** en la parte superior
- Ingresar el token en formato: `Bearer {tu_token_jwt}`
- El token se incluye automáticamente en todas las peticiones subsecuentes
- Permite probar endpoints protegidos directamente desde la interfaz

Beneficios de su Uso

- Documentación Automática:** Se genera desde anotaciones del código
- Pruebas Interactivas:** Ejecutar peticiones sin herramientas externas
- Validación en Tiempo Real:** Ver esquemas de request/response
- Onboarding Rápido:** Nuevos desarrolladores entienden la API fácilmente
- Sincronización Garantizada:** Documentación siempre actualizada con el código
- Generación de Clientes:** Crear SDKs automáticamente
- Testing de Roles:** Probar permisos con diferentes tokens



Estructura Estándar de Respuestas de Error

Componentes ApiError

- **Código HTTP**

Status numérico estándar (400, 401, 403, 404, 500) para identificación rápida del tipo de error

- **Tipo de Error**

Enum descriptivo (BAD_REQUEST, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, INTERNAL_SERVER_ERROR)

- **Mensaje Descriptivo**

Texto en español explicando claramente la causa del error para facilitar debugging

- **Lista de Errores**

Array con detalles de validación por campo cuando aplica MethodArgumentNotValidException

- **Timestamp ISO 8601**

Marca temporal precisa para correlación en logs y auditoría de eventos

Ejemplo de Respuesta JSON

```
{
  "status": 400,
  "error": "BAD_REQUEST",
  "message": "Error de validación",
  "errors": [
    {
      "field": "titulo",
      "message": "El campo título es obligatorio."
    },
    {
      "field": "isbn",
      "message": "El ISBN debe tener formato válido."
    }
  ],
  "timestamp": "2024-01-15T10:30:45.123Z"
}
```

Este formato unificado facilita la integración con frontends y herramientas de monitoreo, asegurando claridad en el manejo de errores.

Casos de Prueba con Payloads JSON

Ejemplos de solicitudes y respuestas reales para validación en Postman, Swagger o herramientas de testing automatizado.

1

Registrar Exitoso

Request:

```
POST /auth/register
{
  "nombre": "luis",
  "email": "luis@gmail.com",
  "password": "12345",
  "roles": [
    "ROLE_ADMIN" o "ROLE_USUARIO"
  ]
}
```

Response 200:

```
{
  "token":
  "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlcyI6WyJST0xFX0FETUIOI0slnN1Yil6lnB1QGdtYWlsLmNvbSIsImhhdCI6MTc2NTUwNzQ4MywiZXhwljoxNzY1NTEyMDgzfQ.jRi-rmk-f_oS65vkoHajPB1MgjuzxgEAlhxi9WYijPE",
  "email": "luis@gmail.com",
  "nombre": "luis",
  "roles": [
    "ROLE_ADMIN"
  ]
}
```

2

Login Exitoso

Request:

```
POST /auth/login
{
  "email": "luis@gmail.com",
  "password": "12345"
}
```

Response 200:

```
{
  "token":
  "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlcyI6WyJST0xFX0FETUIOI0slnN1Yil6ImpAZ21haWwuY29tliwiaWF0Ijo xNzY1NTY0NTgxLCJleHAiOjE3NjU1NjgxODF9.TQsv1juT4WnpZ02nXCGm0-O9JOpJJE-or7Fe0ESTcsk",
  "email": "luis@gmail.com",
  "nombre": "luis",
  "roles": [
    "ROLE_ADMIN"
  ]
}
```

3

Crear Libro (Requiere Token ADMIN)

Request:

```
POST /libros
Authorization: Bearer {token}
{
  "titulo": "Clean Architecture",
  "autor": "Robert C. Martin",
  "isbn": "978-0134494166",
  "totalEjemplares": 10,
  "disponibles": 5
}
```

Response 201: Libro creado con ID asignado y cantidad disponible inicializada.

4

Crear Préstamo

Request:

```
POST /prestamos
Authorization: Bearer {token}
{
  "usuarioid": 1,
  "libroid": 4,
  "fechaPrestamo": "2025-12-12",
  "fechaDevolucionEstimada": "2025-12-12",
  "fechaDevolucionReal": "2025-12-12",
  "estado": "PRESTADO - DEVUELTO"
}
```

Response 201: Préstamo creado con estado ACTIVO y fecha de préstamo registrada, decrementando ejemplares disponibles.

5

Devolver Préstamo

Request:

```
PUT /prestamos/3/devolver
Authorization: Bearer {token}
ID: del prestamo creado (1,2,3,etc)
```

Response 200:

```
{
  "id": 1,
  "usuario": "luis",
  "libro": "stranger",
  "fechaPrestamo": "2025-12-12",
  "fechaDevolucionEstimada": "2025-12-12",
  "fechaDevolucionReal": "AUTOMATICA CON FECHA ACTUAL",
  "estado": "DEVUELTO"
}
```

6

Error de Validación

Request:

```
POST /api/libros
{
  "titulo": "",
  "isbn": "123"
}
```

Response 400:

```
{
  "status": 400,
  "error": "Error de validación",
  "message": "{totalEjemplares=no debe ser nulo, disponibles=no debe ser nulo, titulo=no debe estar vacío, autor=no debe estar vacío}",
  "path": "Validación de campos",
  "timestamp": "2025-12-12 13:47:12"
}
```


Conclusiones y Recomendaciones

Conclusiones

Nivel de Cumplimiento de Objetivos

El sistema **Biblioteca API** cumple exitosamente con todos los objetivos planteados. Se implementó una arquitectura robusta por capas, un sistema de seguridad de nivel empresarial con JWT, y un modelo de datos normalizado que garantiza integridad referencial. La API REST está completamente documentada y es fácilmente consumible por clientes frontend o móviles.

Robustez del Sistema

La combinación de Spring Security, validaciones exhaustivas, manejo global de excepciones y transacciones ACID garantiza un sistema altamente confiable. El uso de soft delete previene pérdida accidental de datos, mientras que el control de acceso basado en roles protege recursos sensibles. Las pruebas realizadas demuestran estabilidad bajo diferentes escenarios de uso.

Mantenibilidad y Escalabilidad

La separación clara de responsabilidades en capas facilita el mantenimiento y evolución del código. La arquitectura stateless con JWT permite escalabilidad horizontal sin problemas de sesión. El uso de Spring Boot y sus convenciones reduce el código boilerplate y acelera el desarrollo de nuevas funcionalidades. La documentación automática con Swagger facilita la incorporación de nuevos desarrolladores.

Beneficios para la Institución

La institución obtiene un sistema moderno que automatiza procesos manuales, reduce errores, mejora la experiencia de usuarios y bibliotecarios, y proporciona trazabilidad completa de operaciones. La inversión en tecnologías estándar de la industria garantiza soporte a largo plazo y disponibilidad de talento para mantenimiento futuro.

Recomendaciones y Trabajo Futuro

1

Implementar Refresh Token

Agregar tokens de actualización con mayor duración que permitan renovar el access token sin requerir nuevas credenciales, mejorando la experiencia de usuario mientras se mantiene seguridad.

2

Sistema de Auditoría de Actividades

Implementar logging detallado de todas las operaciones críticas (creación, modificación, eliminación) con información de usuario, timestamp y datos modificados para cumplir con requisitos de auditoría y trazabilidad.

3

Sistema de Notificaciones

Desarrollar módulo de notificaciones por email o SMS para alertar a usuarios sobre préstamos próximos a vencer, libros disponibles en lista de espera, y confirmaciones de operaciones importantes.

4

Dashboard de Estadísticas

Crear endpoints y visualizaciones para métricas clave: libros más prestados, usuarios más activos, tendencias temporales, tasas de devolución, y proyecciones de demanda para optimizar adquisiciones.

5

Sistema de Reservas

Permitir que usuarios reserven libros actualmente prestados, creando una cola de espera automática que notifique cuando el libro esté disponible.

6

Integración con Sistemas Externos

Conectar con APIs de proveedores de libros, sistemas de catalogación internacional (ISBN), y plataformas de reseñas para enriquecer la información de los libros.

El sistema Biblioteca API representa una solución completa, segura y escalable que establece bases sólidas para la gestión bibliotecaria moderna, con amplio potencial de crecimiento y adaptación a necesidades futuras.